

eboard readme file, devbuild 2026-02-03:

For license information see: license.md or source files, most if not all should contain a license information. The license information displayed in the 'eboardLicenceAndAbout.json' file is the most recent and should be used. In summary: you can use the prototype or the fluidui concept for private or educational purposes or for evaluation of its features. For any other purpose you'll have to contact the project team to get the required authorization. The license will be replaced sometime in the future with an enhanced version.

project team: contact via email: kammel@posteo.de

Project description:

Eboard is a prototype developed using MVVM pattern and C#/WPF. It serves as an experimental platform for research and development of fluid user interfaces, designs and as a tool for testing software concepts.

It can install plugins to enhance its features. It is very flexible in regards of customization. Due to its plugin architecture and rapid ui customization, features can be added swift and designed fast. It has no online functionalities yet, with the exception of linking web site addresses to a plugin. Its state is saved in a bunch of json files with different models. It has a limited set of productivity features for office work due to file and directory links, writing, designing and some features for media consumption of images, music and videos.

Project questions:

How does the ability to change a variety of properties at once in a very limited amount of time affect stress, creativity, concentration, motivation and workflow? What are the minimum required steps per task? What is the maximum of features needed for supporting personal or small to minor art, educational, administrative or business projects or project phases?

Project concept and history:

Since the beginnings in 2022 the goal is to build user interfaces that enable users to change a variety of context areas (CA) to their liking, without limiting the complexity of the features encapsulated, while also being easy to look at and fast to work with.

The idea is that the brain might profit from the ability to change stuff fast and versatile without harming the functional logic needed for a workflow. Therefore eboard and its predecessors allow swift changes of perspective and perception of the work problem and support the user with freedom, visionary harmony experimentation options and an easy achieved feeling of control over the machine. It frees the mind of some stress through the possible changing of the optics of the work problem. Complex, rare moments of free thinking and the more regular creative moments for the invention of new ideas can be captured more easy if basic functions can be accessed fastest and are most flexible. Like a pen and a paper on a desk, where drawing pencils, a scissor and other stuff is present to support spontaneous experimentations with pen and paper during a creative effort for keeping a new idea in the mind long enough for transcription.

The term FluidUI is used since the end of 2025 in the prototype to encapsulate the features that allow changes to CAs and to separate those from other features. Eboard is a successor to AEUI (2024, Aiding Elements User Interface) and YSUI (2022, YRS, Your Startup UI), both developed in C#, using WPF framework, aiming towards the same goal. Eboard follows several fresh approaches while aiming at better code and architectural and logic quality.

In its current state it dwarfs the previous prototypes in some aspects. In return, some features of the previous prototypes aren't a part of eboard yet, like z-index based element visibility, drawing or

complex design element structures.

Project status 2026-02-03:

The previous project status remains mostly unchanged by this build and is part of this document. All other texts have been updated. The code refactoring, renaming and restructuring towards better maintainability, clarity and separation of concerns was continued. Some documentation was added. PluginBaseViewModel, IPlugin interface and FluidUIBaseViewModel have been changed.

The goal was to develop improved plugin architecture and fluid ui features and to get rid off old solutions of earlier development stages. Two new plugin categories were added to the PluginCategory enum. 7 eboard category plugins were developed by encapsulation of CA features or encapsulation of entire non element and non plugin CAs. Some plugins have been moved into other categories. Eboard category is reserved for eboard sdk features and will probably not be accepted for plugins if the assembly is not eboardsdk, but this is not yet implemented.

FluidUI features have been improved, partially debugged and developed:

- Screen CA FluidUI-Stand feature was implemented, it performs definition of minimum and maximum dimensional values (x, y, z) for the screen CA, effecting the range of element CA FluidUI positioning sliders
- Element CA FluidUI-Stand feature was enhanced with skewing (x axis and y axis rotation) transformation, while its x,y and z maximum textboxes have been removed
- Element CA FluidUI-Size feature was enhanced with scaling transformation
- Element CA FluidUI-Design feature was enhanced with LinearGradientBrushes, RadialGradientBrushes and with examples showcasing VisualBrushes; VisualBrushes are not yet permanent and will be replaced by a default VisualBrush on program start
- Element CA FluidUI-Data was enhanced with a tooltip option and with changes to the FluidUI tab, it is now not only possible to save and load FluidUIContext models to or from file, but to copy and paste contexts between CAs; subcontext selection can be used to save or copy or paste or load or apply the entire FluidUIContext or only subcontexts
- drop event for FluidUIContext model files (*.fcf) has been added to all CAs, but subcontext selection has no effect yet
- one FluidUIContext can be saved in MainWindow CA and pasted onto any CA, the plan is to improve this feature, so that newly instantiated element and screen CAs derive their FluidUIContext upon creation from this template copy; now it is possible to copy any FluidUIContext from any CA to this property using FluidUI-Data-Fluid UI-Save Configuration copy button and paste its value as deepcopy on any CA or element CA using FluidUI-Data-Fluid UI-Load Configuration paste button, the copy is permanently saved and will be loaded on start, it can be changed by copying another FluidUIContext or reset using the MainWindow CA right-click context menu item Eboard – Reset FluidUI default

CA right-click context menus have been improved, partially debugged and developed:

- MainWindow(eboard) CA right-click menu now consists of 4 submenus, one for mainwindow, one for screen, one for FluidUI and one for eboard options
- Navigation CA right-click menu now consists of 3 submenus, one for panel, one for screen and one for FluidUI options
- Screen CA right-click menu now consists of 4 submenus, one for screen, one for screen change, one for FluidUI options and one for invocation of eboard category plugins (e.g. if the user does not want to use (or see, using MainWindow CA right-click MainWindow-Change option to hide the controls) the mainwindow controls in the mainwindow CA, but needs them now and then)
- Element CA right-click menu now consists of 2 or 3 submenus depending on plugin setup, one optional submenu for plugin menus, one for element and one for FluidUI options

Elements can now be duplicated (instant creation of n copies on the same screen coordinates) and copied or moved and pasted to the same or a different screen:

- storage and display of copied elements is a job of the MainWindow CA, which has an option to deactivate copy list content display (right-click menu, Eboard-Display copy list)
- element copy list is emptied after pasting, it is planned to develop a copy list plugin, that allows for managing the contents of the copy list
- element copies will be stored on program exit and restored on start so that pasting can be done any time
- an element copy is a PluginCopy object that should hold only most needed data of IPlugin
- the FluidUIContext of a copy is a deep copy of the FluidUIContext of the original element
- plugin models can be copied using the new IPlugin.PluginModel object
- existing plugins with content have been upgraded with the new system, so if you duplicate or copy an image or link plugin, the model state of the plugin will be copied as well
- some bugs remain, not all models will be copied in the current build, only a handful of plugins could be tested so far

Plugin improvements:

- plugins can now set a usercontrol and a viewmodel for a plugin menu in the surrounding element CA, the initial goal of a menuitem control and a viewmodel could not be achieved in this first attempt
- plugin developers can now set up copy constraints in addition to instantiation policies
- StandardText plugin menu was enhanced
- Image was enhanced with an experimental ratio feature that will be reworked later
- Link plugin menu was enhanced with an extended tooltip option
- parts of the new LogOutBar and MenuBar plugins can be hidden, the state of the model will be saved and restored
- Areas have been improved and can now hide their controls, but they are more limited than before, which will be fixed in upcoming builds, because right-click menus of plugins are deactivated for most plugins; it is planned to use plugin right-click menus in areas for each plugin within the area, so that they are again changeable within the area
- Polygon shape and PolygonMaker have been added and can be used for creation of polygons, PolygonMaker is easier to use, it offers a surface in foreground brush color where the user can click anywhere to create points for the point collection; it can use Coordinates plugins to create polygon points from their coordinates; it can create Coordinates plugins from the point collection; the point collection can be edited and the items can be moved via drag and drop to alter their position in the list, it is planned to highlight dragged points somehow or even allow for dragging of points on the polygon click surface; be advised: calculation of polygons affects ram usage, polygons also can have extreme object sizes with a lot of invisible area, that might block elements below the polygon from being clickable
- ellipse shape saw the beginning of shape plugin development continuation, but progress was slim, it got options to show or hide the surrounding element and to alter its stroke thickness, but is unfinished
- shape plugin right-click menus are mostly broken atm, the next build or the one after will probably contain fixed shapes and fully developed shape menus

MainWindow CA now shows some messages upon user activity as well as an indication if copies are stored in the copy list, it is planned to enhance the messaging feature in future updates and to implement an option to hide the output altogether; About plugin and Manual plugin can now be displayed in MainWindow CA if no screen is active

Some bugs and halfway implemented features have been fixed:

- deletion of screen and element selections should work again
- FluidUI configuration selection, reset, as well as save and load should work

Project status 2026-01-13:

The prototype underwent some major changes to its codebase in the recent weeks. Although some parts have been refactored into fewer lines of code and better separation of concerns, many areas of the source are not yet refactored or robust. Some features have been added that require further development and testing, but they showcase what they are about in an ok manner so far. Most if not all visible buttons should have the described effects and should work. Some menus use sliders and some of them need the hit of the nearby OK button to have effects.

The current state of the project is work in progress. There is no versioning besides dates. Some of its features are training exercises or experiments. Others are hacked as mockups until better understanding or implementations are at hand or needed. The main goal is researching the concept itself. For now imagination must serve, where the proves of concept of the mockups within the prototype end.

Next steps will include further refactoring the source code, improving memory usage, building better xaml styles (this requires some research into the topic and is low prio, because not that important at this stage), continuation of feature development and modularization of the project and plugin architecture, as well as basic drawing and command features, maybe even first mockup plugins for database access or server interaction.

Of lesser priority are small improvements to existing features, fixing non critical bugs, code documentation and better integration of logging and exception handling into the code, fixing or adding xaml styles to ui elements like sliders, listbox items, comboboxes and tabitems, which are still mostly on default style and are not yet affected by fluid ui changes, which can lead to color combinations that are unsuited to see the information within the default styled control.

Features:

→ context areas:

MainWindow(eboard), Eboard Browser, Screens and Elements are fluid ui context areas and will be explained further down below. Any other control or plugin can have context menus depending on the authors choices. Right clicking on any surface element will open a context menu. If the context is part of the fluid ui, it will contain a FluidUI submenu, which contains 4 to 5 submenus for alteration of subcontext properties.

→ FluidUI:

FluidUI menus will differ a little bit depending on the context area and the function it serves. All of them support changes to the items 1-4 below:

- 1) Design: change opacity for the CA or brushes for background, border, foreground and highlight. Highlight brush serves for indication of selection and for warnings to the user. Supported are SolidColorBrushes, LinearGradientBrushes, RadialGradientBrushes and ImageBrushes. There are some examples for VisualBrushes, but they are not saved on program exit.
- 2) Font: change of font family, font weight and font size.
- 3) Size: change of margin, padding, border thickness, corner radius, scale, width and height of the context areas outer border.
- 4) Data: change of context title, description text, index text list, key text list and a total of 32 index and key text properties (the lists and 2x4xquads are experimental. The lists could be useful for internal logging activity or as a protocol/diary, the quads as storage mockup for potential properties like credentials, access or id keys, tokens and the like.). Data offers some functions for FluidUI configuration like tooltip toggle or configurable FluidUIContext copy and paste, as well as save and

load.

5) Stand: if supported by the context area: change of position and rotation or change of boundaries

→ **MainWindow(eboard) CA:**

The mainwindow context area is the outmost control and encapsulates all other context areas. It contains several eboard category plugins and some unique controls. It has no window caption. It can be left-clicked and dragged if the clicked surface does not contain clickable areas like buttons or scrollable areas like a screen with height greater than the mainwindow CA and has an opacity or an alpha value(for brushes) above 0. Changes to its FluidUIContext will affect all of its controls, except for other context areas. MainWindow CA plugins have an inner region, that derives graphic properties from MainWindow CA and an outer region, that derives from the element CA. This can, given a distinguished design, help to identify main program functions fast and precise.

It contains the following controls:

- Eboard Title: center top textbox, shows the title of the mainwindow context area
- MenuBar: plugin, consists of:
 - Eboard Browser: togglebutton, keeps Eboard Browser context area visible or hidden
 - Screen Control: togglebutton, keeps Screen Control ui visible or hidden, Screen Control UI is defined by the active screen CA FluidUIContext
 - plugin selection menu: menu, shows available plugins, each can be instantiated onto the screen if a screen is selected, more about plugins and screens will follow further down below, plugin selection menu UI is defined by the active screen CA FluidUIContext, if no Screen CA is selected in Navigation CA plugin selection menu will be hidden
- LogOutBar: plugin, consists of:
 - Manual: button, instantiates a manual plugin element on the selected screen or on the MainWindow CA if no Screen CA is selected in Navigation CA
 - About: button, instantiates an about plugin element on the selected screen screen or on the MainWindow CA if no Screen CA is selected in Navigation CA
 - Shutdown: button, opens dialog and initiates hardware shutdown upon confirmation
 - Off: button, opens dialog and initiates eboard shutdown upon confirmation
- Eboard Browser: plugin, Navigation CA, visible if Eboard Browser button is checked
- Screen Control, plugin, screen CA options, visible if Screen Control button is checked
- message strip: textblock, displays messages in highlight brush
- Screen CA: user control, visible in center if a screen is selected in Navigation CA
- ScreenChanger, plugin, has buttons for change of selected screen and a screen title textbox
- copy list indicator: border with tooltip, both become visible if plugin copy count in element copy list is above 0, the tooltip displays the contents of the copy list
- tooltip: tooltip, displays FluidUI-Data text property
- context menu: right-click menu, contains FluidUI menu and MainWindow CA functions

→ **Navigation CA:**

The EboardBrowser is located top left below the menu line if visible or anywhere on a screen if instantiated as plugin. It consists of a panel on the left that gives an overview over existing screens, two buttons in the center, that allow hiding or showing the left or right area of the browser element and the right area, that is for details about the active screen and shows some data. A tooltip in the bottom half of the right area shows some statistics about the active screen. Changes to its FluidUIContext will change all its controls except for the representations of the screens that use the FluidUIContext data of the screen they represent. Navigation CA can be fully instantiated as plugin on a Screen CA with the eboard category plugin EboardBrowser or partially with the eboard category plugin ScreenDataBoard. It contains the following controls:

- Panel: panel, shows screen representations in a panel within a scrollviewer
- center buttons: two buttons, left button will hide or show the panel on the left, right button will hide or show the ScreenDataBoard plugin on the left

- ScreenDataBoard: plugin, consists of:
 - Add: button, adds a new screen to the panel with the values from the input field
 - Edit: button, apply the values from the input field to the active screen
 - Delete: button, shows dialog and deletes selected screens after confirmation
 - Textblocks and TextBoxes between the buttons that display name, depth, width, height and opacity of either the active screen and are used for definition of new screens
 - data board: displays screen panel list position and FluidUI-Data title and text properties
 - data board tooltip: displays statistics(creation date and plugin count) of active screen, visible only above data board
- tooltip: shows FluidUI-Data text
- context menu: right-click menu, contains FluidUI menu and Navigation CA functions

→ **Screen CA:**

The screen context area is located in the center of the MainWindow CA. Screens encapsulate empty virtual space and can be of any size and depth. Changes to its FluidUIContext will affect its empty space, border, context menu, the plugin selection menu and the Screen Control UI. Screens are at the moment the only valid context area targets for element instantiation. It is planned to explore docking panel for the mainwindow context, so that plugins could be dragged onto e.g. the mainwindow context or certain areas outside the screen CA. At the moment only screens support elements. These can be placed anywhere within the screens area and even beyond, if the user chooses to do so by entering coordinate values into the elements FluidUI-Stand subcontext. ScreenControl UI targets the active screen and has options to manipulate its elements. Screen CAs contain the following controls:

- Panel: for its element collection
- 0 – n elements, each is a unique element CA that encapsulates a plugin
- tooltip: showing FluidUI-Data text and screen position data
- context menu: right click menu, contains FluidUI menu and Screen CA functions

→ **Element CA:**

Element context areas are encapsulated by a screen context area. Each element can be moved within its encapsulating context via left-click and drag. Each element has a z property for its depth position on the screen and can be rotated. Elements get a z value of 1000 if they are moved, which means that your mouse cursor will hit any element with a higher z than that. It is possible to select elements. Selected elements can be manipulated by using the Screen Control UI or via an elements right-click context menu. During the most recent refactoring some if not all right-click context menu group functions like group placement, group rotation and so on have been disabled until there is time to adapt their logic to the new underlying system. Selection movement also got buggy during the process. An element CA contains the following controls:

- plugin CA: control, encapsulating a feature, a function or a complex software
- tooltip: showing FluidUI-Data title and text, as well as screen position and plugin type
- context menu: right click menu, containing FluidUI menu and element CA functions

→ **Plugin (CA):**

Plugins are encapsulated by Element CAs and can either use the elements design or, given the WPF framework underneath, have their own (and become their own fluidui CA). IPlugin interface offers a FluidUI property for the plugin. Plugin authors can choose whether a plugin context menu is required or not. Same holds true for tooltips. It is planned but not yet implemented to restrict the plugins access towards eboard during runtime to a limited amount of getter functions or properties and a limited amount of setting functions or properties.

Each plugin main view model class must derive from a child of PluginBaseViewModel or inherit the abstract PluginBaseViewModel class implementing IPlugin interface to ensure it can be

detected by and processed within the eboard prototype.

Each plugin must provide a data template for its views and viewmodels in the resource dictionary that is referenced in the viewmodel inheriting `PluginBaseViewModel`. Each plugin must specify its constraints or the plugin won't be instantiated or copied and each plugin can specify a menuitem and a menuitem viewmodel if they need such.

All dll-files within `Eboard\Plugins\` are searched for the appropriate base type. It is quite easy to add plugins to the existing prototype. Right now plugins can be installed by copying a proper dll file into the `Plugins\` folder that is below the `Eboard\` folder in the directory that contains `eboard.exe`. Eboard only reads from the folder at program start. It is planned to improve the current system, so that plugins can be installed and deleted or deactivated with greater comfort during program runtime, which was already tested successfully, but requires some planning and some changes.

Plugins are grouped into categories. The definition for the categories is not layouted in detail yet. A plugin can contain a simple function or offer a small set of functions and features like linking files. It could also be complex and feature rich. For now, the Addon category plugins serve as mockups for complex software. Depending on the setup within the implementation deriving from `PluginBaseViewModel`, plugin instantiation can be restricted. Of all the values of the enum, only a few are processed right now (e.g.: `Unique` and `Global` have no effect).

Uptime plugin and Drives e.g. have a `OnePerScreen` instantiation policy. `MainWindow CA` message strip shows the instantiation policy of a plugin upon plugin instantiation and the copy constraints and instantiation policy upon any copy or paste activity. `CopyConstraints` impact element CA options and will show or hide certain submenu functions accordingly. Copy constrained plugins can't be copied, moved or pasted. In case they are part of a selection that gets copied, an error message will be displayed in `MainWindow CA` message strip and the plugin will be ignored in the copy process.

Any plugin with an unconstrained instantiation policy can be instantiated, copied or pasted an unlimited amount per screen. Each element holds its own instance of its plugin type main viewmodel. A plugin contains the plugin controls, whatever they might be. Plugins can use the save and load methods of the `pluginbaseviewmodel` to store content (or trigger saving and loading mechanisms). In order to use the implemented functionality, a json serialization compatible model class is required.

Summoner plugin allows to circumvent instantiation constraints. There is no decision yet if this will remain or subject to change in the future.

→ **Plugin features:**

Standard text(for creating and editing small texts, saves and loads stf named json files), basicAV(for videos, audio files or displaying pictures), image(for linking an image), link(for linking files, directories or websites), areas(lines and rows of plugin types), summoner(direct plugin instantiation), shapes(basic shapes, path, polygon and text shapes for drawing and design), uptime(clock, date and system runtime), coordinates(plugin and mouse screen coordinates, can be used for polygon creation), drives(hdd/sdd/usb/cd drive access), BudgetWatcher(simple budget management), protocol(simple diary or note), plugin manager(plugin management, later intended for installation and deinstallation of plugins), about(displays license, version and concept information), manual(displays help texts, later maybe pdfs, flow documents or html or web resources), mynote and soundmix are mockups of more complex iterations of basicAV and protocol, countdown timer(set up a countdown), polygonmaker (for creation of polygons), eboard menu plugins (`MainWindow CA` and `Navigation CA` features as plugins, `Screen CA` plugin is

planned), Kraken (serves as a development tool, its task is to be an external plugin loaded during runtime and access everything possible within the sdk and the application in order to read and write stuff from or into eboard during runtime, as if it were an external developed plugin with bad intentions in mind; in its earliest implementation all it does is accessing some public properties and displaying the findings into a textbox, but it already showed vulnerabilities like undesired access to some properties; it will help to highlight quirks in the design that have to be fixed in upcoming updates).

The other plugins have no functionality besides being mockups for more plugins of any kind.

→ **selection:**

Each element can be selected via shift + left mouse click or via Element CA right-click context menu item Element-Select. A group of elements can be selected using Screen Control UI by setting up the selection extent and hitting the select button. Title select option will search for any element context area with the given title {broken}. Specific will show all plugins and allow the selection of instances of a certain type. The other settings will select all elements or the given category, or help narrowing down the plugin amount {Eboard and System category selection is broken}.

Deselection can be done by clicking anywhere within the mainwindow context area for all selected plugins (given the underlying surface is not a button, a scrollable area, a menu or a menu item) or via Element CA right-click context menu item Element-Select. It is planned to integrate more selection and filtering features.

→ **permanence:**

Each FluidUI CA and any content of plugins using the PluginBaseViewModels Save() and Load() methods will be saved upon program exit and sometimes upon crash. All successfully saved data will be loaded upon program start and for some features upon user choice. It is planned to offer customization for eboard data storage locations. A savepath can be set using the right click menu of the mainwindow CA (Eboard-Save Path), but the feature was not tested after the most recent refactoring and development work.

FluidUI CA data is saved in a variety of *.edf files (for CAs) or *.fcf files (for FluidUIContext models). Plugin content is saved in *.ecf files. The extensions hide a json file with a certain model underneath. Right now eboard only saves texts, links and FluidUIContext data to harddrives and restores its state from those files. It is planned to improve the existing permanency infrastructure in the future. For fastest access to data linked by and presented with eboard, local storage should remain. In the future, backups could be written to a cloud or loaded from if local storage is corrupt or missing.

→ **customization:**

Customization can be achieved using FluidUI context menu for each CA. FluidUIContext models can be saved to and loaded from harddrives. *.fcf files storing a FluidUIContext can be dragged and dropped on any FluidUI CA and change them instantly to an entirely different look. In a CAs right-click menu (FluidUI-Data-Fluid UI) saving, copying or loading and pasting of FluidUIContext models or parts of such models can be configured and done. Subcontext models that are unset will be stored as null value and ignored when a FluidUIContext is applied to another.

'Load Configuration' tab options apply a FluidUIContext (or parts of it) from file or the MainWindow CA FluidUIContext copy property to the CA. 'Save Configuration' tab options store a FluidUIContext (or parts of it) model either on the selected hard drive location or as copy within MainWindow CA FluidUIContext copy property.

For selections of elements it is possible to load or reset FluidUI contexts using Screen Control UI Fluid UI tab. They will be loaded or reset for every element of the selection. The system is a first draft, some bugs are already known and noted. FluidUI menu in Screen Control is WIP and only partially operational atm.

The prototype is quite flexible regarding customization. Due to the FluidUI features any plugin or CA can become a piece of art, a sleek piece of corporate design or a mockup for planned stuff. In education, element configuration options could either be taught or used to support learning. It could be done to create an image with proper dimensions in order to use it as Border image e.g. for a text element, so that vital knowledge or the goal in question can be directly seen (but not changed) besides the work area, where concentration is bound until the task is finished.

The *.edf or *.fcf files can be comfortable edited with code editors, allowing for fast changes before starting eboard or loading FluidUIContext files or from within eboard using the FluidUI context menu. Existing FluidUIContext files can be dragged and dropped on any CA and will change the CA they are dropped on. Visual brushes and drawing features are planned or in development. If you develop your own plugins, you can customize eboard even further.

Mainwindow CA controls can be hidden, tooltips can be deactivated under FluidUI – Data - Title and Text. Each context area can be named and colored differently. So if you have 2 screens with 5 plugins each, you could change a total of 12 context areas, including Eboard Browser and MainWindow CA to your needs within a few clicks. With the new save and load of FluidUIContext models and the new selection features customization has become even faster.

In order to customize eboard to your project needs, set up some screens, give them names if you wish and organize files, images, texts or mediafiles according to your needs. Create designs that you like or want to try out and apply them onto plugins of any kind. FileLinkArea plugin offers a multiclick button that allows to open all linked files at once.

→ **mousewheel:**

With mousewheel you can change an elements z value. Ctrl + mousewheel changes an elements rotation. Mousewheel rotation and z-index alteration will only work if the screen CA has no vertical scroll bars. In such cases, until the underlying issue can be targeted and fixed, a workaround is to use the right-click context menu FluidUI-Stand submenu and change the rotation or z-index via slider or via inserting values into the textboxes left to the sliders.

→ **leftclick:**

On some controls that display text ctrl+leftclick can be used to change the control, so that it allows for a change of the text. On Link and Image plugin ctrl+leftclick can be used to reset links. Double left click on mainwindow context will trigger showing or hiding of all mainwindow controls except the active screen.

Final words:

Be advised, it is very likely that PluginBaseViewModel, IPlugin interface and FluidUIBaseViewModel will be subject to changes like renaming, deletion or addition of properties and methods in future updates. The concept is a research in progress. Until final decisions for plugin architecture have been achieved, you will have to recompile your plugin dlls if changes to the plugin architecture occurred.

I deleted the Gold element link. I didn't include the picture because i am unsure about the license. If you want to see golden surfaces you will have to replace the link with a new one and download a picture yourself. Gold plugin could be used for any valueable picture, not just shiny metal. It is not

really necessary atm given the possibilities of the CAs.

Loading procedure is atm a serial process (parts of it could be run in parallel), resulting in longer loading times on higher plugin count upon program start. Some calculations could be done outside the ui thread. Memory usage is not optimized and will result in higher memory usage upon repeated screen changes, plugin duplication and the like. Polygon calculation and creation seems to be quite ram consuming. Garbage collection works to an extend, but takes a while to free ram. Later builds probably will target memory consumption, program execution speed and the like. For now the prototype does an okay job, given that other aspects of the fluid ui research are more important right now.

If it is used as a project starter tool memory usage and speed won't matter anyway. After setting up instant access to relevant files or folders via Link or FileLinkArea plugin in a logical or graphical manner using screens, colors or whatever, changes to UI or switching screens will probably be limited to a few times per work session or only to the setup phase.

If it shall be used as a tool for creation of graphics or designs or whatever, memory usage and speed will matter. The prototype is only capable to serve such needs in a limited amount at its current stage. Further optimizations and plugins containing special workflows would be required for such.